

CS101C: Introduction to Programming (Using C)

Instructors: Prof. Aikta Arya and Prof. Konjengbam Anand

CSE Department

Indian Institute of Technology, Dharwad

Week 10

Today's Class (12/10/2026)

File Handling!!

Why do we need file handling?

The disappearing marks problem!

1 We write a simple C program.




```c
#include <stdio.h>

int main(void)
{
    int marks = 95;
    printf("My marks = %d\n", marks);
    return 0;
}
```

File Handling!!

Why do we need file handling?

The disappearing marks problem!

1 We write a simple C program.




```c
#include <stdio.h>

int main(void)
{
    int marks = 95;
    printf("My marks = %d\n", marks);
    return 0;
}
```

2 We run it...

Output

```
My marks = 95
```



File Handling!!

Why do we need file handling?

The disappearing marks problem!

1 We write a simple C program.




```c
#include <stdio.h>

int main(void)
{
    int marks = 95;
    printf("My marks = %d\n", marks);
    return 0;
}
```

2 We run it...

Output

```
My marks = 95
```



3 We close the program and run it again.

Output

```
My marks = 95
```



File Handling!!

Why do we need file handling?

The disappearing marks problem!

1 We write a simple C program.




```c
#include <stdio.h>

int main(void)
{
    int marks = 95;
    printf("My marks = %d\n", marks);
    return 0;
}
```

2 We run it...

Output

```
My marks = 95
```



3 We close the program and run it again.

Output

```
My marks = 95
```



4 The problem...

The statement assigning 95 to marks executes every time the program runs. So we always get 95.

Run program	Run again
→ marks = 95	→ marks = 95
→ print 95	→ print 95

But we actually want to enter marks once, save them, and retrieve them next week.



File Handling!!

Why do we need file handling?

The disappearing marks problem!

1 We write a simple C program.




```c
#include <stdio.h>

int main(void)
{
    int marks = 95;
    printf("My marks = %d\n", marks);
    return 0;
}
```

2 We run it...

Output

```
My marks = 95
```



3 We close the program and run it again.

Output

```
My marks = 95
```



4 The problem...

The statement assigning 95 to marks executes every time the program runs. So we always get 95.

Run program
→ marks = 95
→ print 95

Run again
→ marks = 95
→ print 95

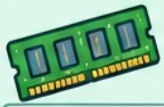
But we actually want to enter marks once, save them, and retrieve them next week.



5 Where does data live?

Variables live in memory while the program is executing.

Memory (temporary)



marks = 95

Lost when the program ends.

We need files!

File (persistent)



marks: 95

(on disk)

Stays even after the program closes.

File Handling!!

Why do we need file handling?

The disappearing marks problem!

1 We write a simple C program.




```c
#include <stdio.h>

int main(void)
{
    int marks = 95;
    printf("My marks = %d\n", marks);
    return 0;
}
```

2 We run it...

Output

```
My marks = 95
```



3 We close the program and run it again.

Output

```
My marks = 95
```



4 The problem...

The statement assigning 95 to marks executes every time the program runs. So we always get 95.

Run program
→ marks = 95
→ print 95

Run again
→ marks = 95
→ print 95

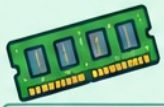
But we actually want to enter marks once, save them, and retrieve them next week.



5 Where does data live?

Variables live in **memory** while the program is executing.

Memory (temporary)



marks = 95

Lost when the program ends.

We need files!

File (persistent)



marks: 95

(on disk)

Stays even after the program closes.

6 The central problem

Variables normally store data in memory while the program executes.

We need **files** to preserve data **beyond a program's execution**.



Memory (temporary) → File (persistent)

Same data... Longer life!



That's why we need **file handling!**

From temporary memory to permanent storage.

Memory vs. Files — Temporary vs. Persistent Data

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड

RAM / Variables

RAM
Temporary workspace

↓

Program terminates

Temporary data is lost

- Useful for calculations.
- Stores variables during execution.
- Usually does not preserve variable values for the next execution.

Example

```
int marks = 95;
```

Files / Persistent Storage

`marks.txt`
Saved on storage

↓

Program terminates

Data can remain saved

- Keeps data for later use.
- Can be read or modified later.
- Useful across multiple executions.

Example

`marks.txt` stores 95

A File

What exactly is a file?



A file is a **named collection** of data stored on a **storage device.**



(hard disk, SSD, USB, etc.)

For example...

`marks.txt` might contain:

marks.txt

```
Aman 85
Priya 92
Rahul 78
```

This is a text file with data stored in it!

A C program can:

1 Open the file.

```
FILE *fp;
fp = fopen("marks.txt", "r");
```



File is opened for reading ("r").

2 Read data from it or write data to it.

C program

```
char name[20];
int marks;
while (fscanf(fp, "%s %d", name, &marks) != EOF) {
    printf("%s %d\n", name, marks);
}
```

Reads, processes, or updates data

3 Close the file.

```
fclose(fp);
```



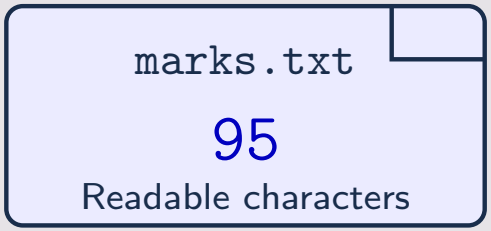
File is closed and data is saved.

A file gives your data a home — beyond the life of your program!

Text Files vs. Binary Files



1. Text File



How is data represented?

As readable characters.

Example Content

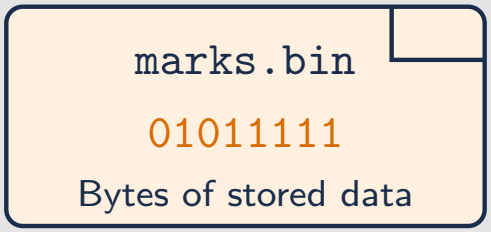
```
95
```

Common uses

- Reports and log files
- CSV files
- Configuration files

Usually easy to inspect in a text editor

2. Binary File



How is data represented?

As bytes in a particular representation.

Possible Representation

```
Bytes: 5F 00 A2 ...
```

Common uses

- Images and audio
- Compact data records
- Serialized data

May not be readable in a text editor

A file extension such as `.txt` or `.bin` does **not** determine how C interprets the data. **The program's file operations and data representation matter.**

The Mysterious FILE *

Before Opening a File...

We need to understand one unfamiliar declaration:

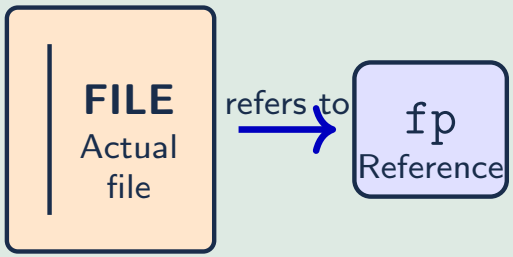
```
FILE *fp;
```

`FILE` A type declared by the C standard input/output library.

`*` Indicates that `fp` is a pointer.

`fp` The variable name.

Think of a Library!



The file is like a book; `fp` is the reference used to access its stream.

Don't Forget the Header!

Before using `FILE`, write:

```
#include <stdio.h>
```

This header provides the declarations needed for standard file operations.

The Mysterious `FILE *`

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Important Distinction

- `fp` is not the file contents.
- Its printed address is not the file's location on disk.

Remember in One Sentence

`FILE *fp;` declares a pointer used to refer to a file stream managed by the C library.

Open Your First File

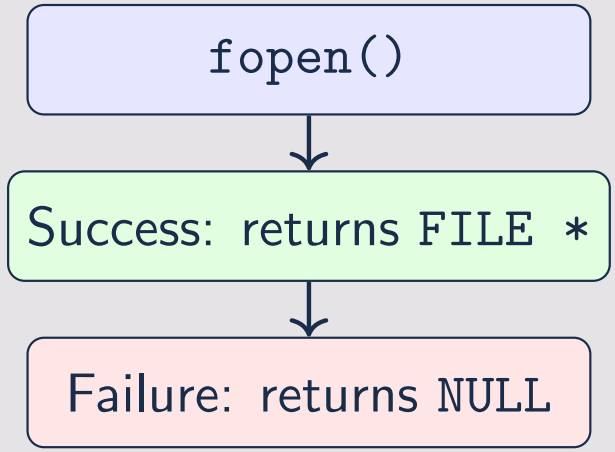
```c
#include <stdio.h>
int main(void)
{
    FILE *fp;
    fp = fopen("book1.txt", "r");
    if (fp != NULL)
    {
        printf("File opened successfull
        fclose(fp);
    }
    else
    {
        printf("Could not open the file
    }
    return 0;
}
```

What Does "r" Mean?

"r" means open an existing file for reading.

Step 1: Open the File

`fopen("book1.txt", "r")`



- Argument 1: filename
- Argument 2: opening mode

Step 2: Check and Close

`fp != NULL`
The stream opened successfully.

`fclose(fp);`
Closes the stream and releases

Open Your First File



```c
#include <stdio.h>
int main(void)
{
    FILE *fp;
    fp = fopen("book1.txt", "r");
    if (fp != NULL)
    {
        printf("File opened successfully!\n");
        fclose(fp);
    }
    else
    {
        printf("Could not open the file.\n");
    }
    return 0;
}
```

Why Must We Check?

The file may not exist, or access may be denied.

Never assume that `fopen()` succeeded!

Predict the Output!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

The Situation

Assume `book1.txt` does not exist in the program's working directory.

What Will This Code Print?

```
FILE *fp = fopen("book1.txt", "r");
if (fp == NULL)
    printf("A");
else
    printf("B");
```

Choose Your Answer

A. `A`

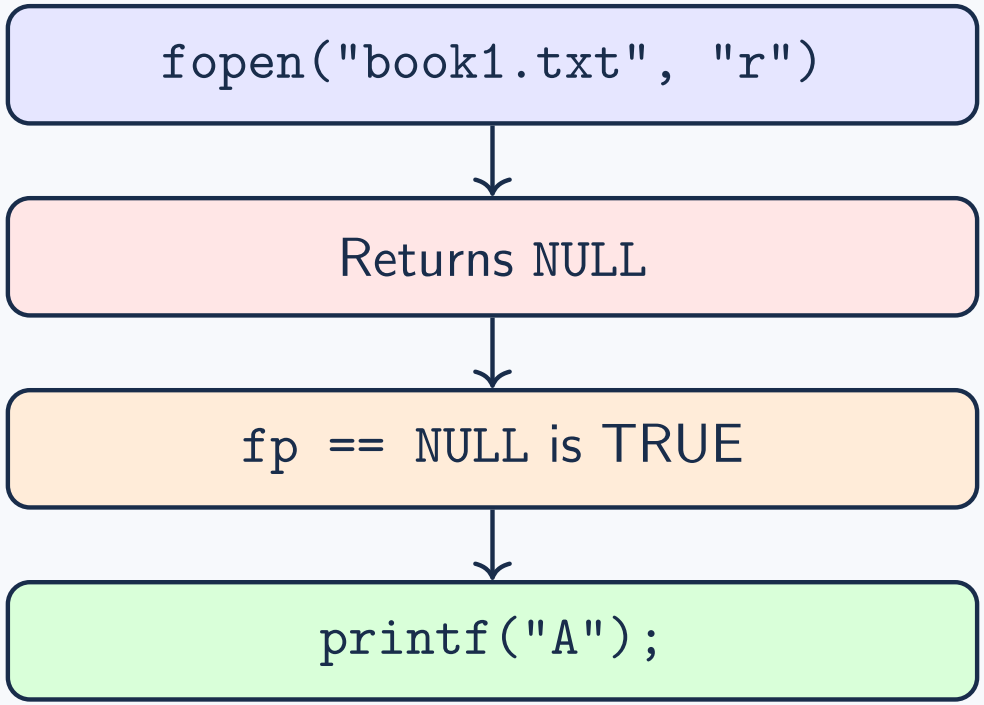
B. `B`

C. Nothing is printed



Correct Answer: A — Prints A

When `book1.txt` does not exist, `fopen("book1.txt", "r")` cannot open it and returns `NULL`.



Why Not B?

The `else` branch executes only when `fp != NULL`, meaning the file was opened successfully.

Today's Class (14/10/2026)

File Modes: The Door Decides!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड

`fopen("marks.txt", MODE)`
The **mode** tells C what we want to do with a file.

`"r"`	`"w"`	`"a"`
← READ	× WRITE	+ APPEND
Read existing data	**Write from scratch**	**Add new data**
• File must exist. • Starts at beginning. • Fails if missing.	• Creates if missing. • Erases old contents! • Starts writing afresh.	• Creates if missing. • Preserves old contents. • Writes at the end.

Remember the Three Doors

`"r"` = Read `"w"` = Wipe and write `"a"` = Add at end

File Modes — Beware of "w"!

Suppose `marks.txt` already contains 100 student records. What happens when we execute this statement?:

```
FILE *fp = fopen("marks.txt", "w");
```

File Modes — Beware of "w"!

Suppose `marks.txt` already contains 100 student records. What happens when we execute this statement?:

```
FILE *fp = fopen("marks.txt", "w");
```

Before Opening

Aman: 85
Priya: 92
Rahul: 78

`marks.txt` *97 more records...*

Old data exists

After Opening with "w"

`marks.txt` **EMPTY** Old contents removed

Data truncated immediately

What Does Truncate Mean?

The existing file is reduced to zero length when opening succeeds. New data can then be written to it.

Never use "w" when you want to preserve old contents!

File Modes: What Does the Letter b Mean?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

The Simple Rule

Adding `b` selects **binary mode**.

`"r"` → `"rb"` `"w"` → `"wb"` `"a"` → `"ab"`

Mode	Purpose	Missing file	Existing file
`"r"`	Read text	Fails	Reads from start
`"w"`	Write text	Creates	Truncates
`"a"`	Append text	Creates	Writes at end
`"rb"`	Read binary	Fails	Reads binary stream
`"wb"`	Write binary	Creates	Truncates
`"ab"`	Append binary	Creates	Writes at end

Text Mode

`fopen("report.txt", "r");` Useful for readable text, reports and logs.

Binary Mode

`fopen("photo.bin", "rb");`
Useful for reading binary data as bytes.

File Modes — What If We Need to Read AND Write?



The Plus Sign: +

+ means the stream supports both reading and writing.

Mode	Meaning
`"r+"`	Read and write; file must already exist.
`"w+"`	Read and write; creates a file or truncates an existing one.
`"a+"`	Read and append; creates the file if needed. Writes go to the end.
`"rb+"` / `"r+b"`	Read and write a binary stream; file must exist.
`"wb+"` / `"w+b"`	Read and write a binary stream; creates or truncates.
`"ab+"` / `"a+b"`	Read and append a binary stream; creates if needed. Writes go to the end.

Compare These Carefully

- `"r+"`: preserves existing contents initially.
- `"w+"`: destroys existing contents when opening succeeds.
- `"a+"`: preserves existing contents and appends writes.

Quick Challenge!

A file named `scores.txt` already contains:

```
Aman 85
Priya 92
Rahul 78
```

We execute:

```
FILE *fp = fopen("scores.txt", "a");
fprintf(fp, "\nNeha 95");
fclose(fp);
```

Assume opening succeeds. What will the file contain afterward?

A. Only `Neha 95`

B. The original three records, followed by `Neha 95`

C. The file becomes empty

D. Opening always fails

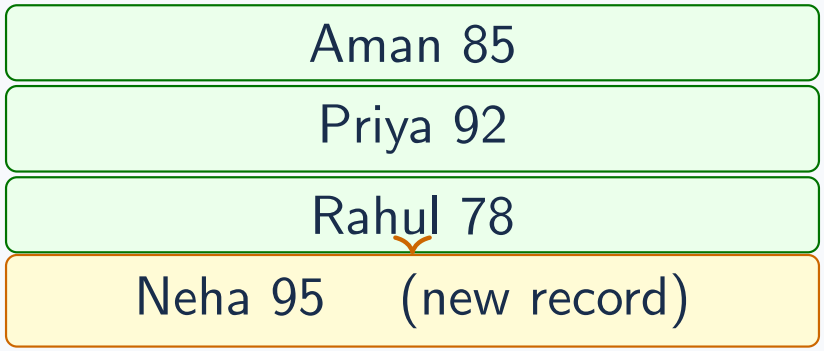
Quick Challenge — Answer

Correct Answer: B

Mode `"a"` preserves the old contents and directs writes to the end of the file.

Resulting File Content

```
Aman 85
Priya 92
Rahul 78
Neha 95
```



Three Modes to Remember

`"r"` = Read `"w"` = Wipe and write `"a"` = Add at the end

Experiment: "w" vs. "a"

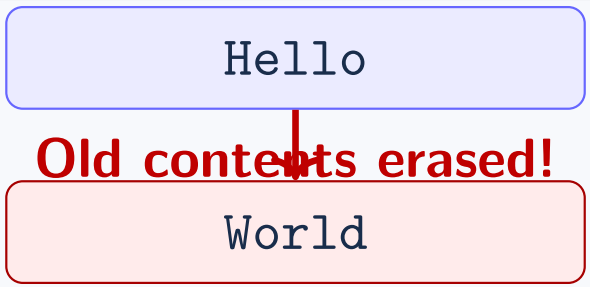
Starting Point

Imagine notes.txt already contains:

Hello

Program A: Mode "w"

```
FILE *fp =
    fopen("notes.txt", "w");
fprintf(fp, "World\n");
fclose(fp);
```

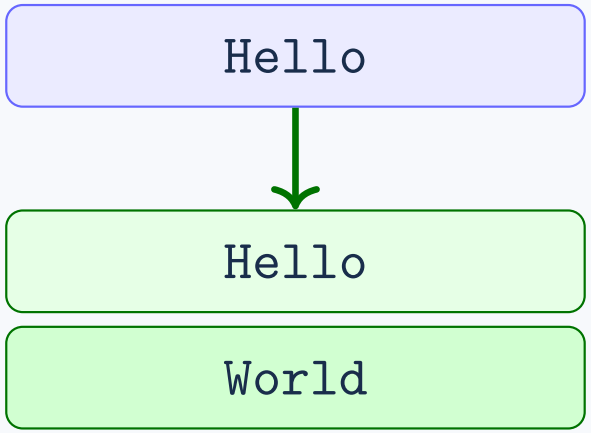


Replace

Program B: Mode "a"

```
FILE *fp =
    fopen("notes.txt", "a");

fprintf(fp, "World\n");
fclose(fp);
```



Append

What Is Actually Saved?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Program A — "w"

Final contents of `notes.txt`

```
World
```

Old contents removed

Only `World` remains.

Program B — "a"

Final contents of `notes.txt`

```
Hello
World
```

Original + new text

Both words remain.

The Rule to Remember

`"w"` = Start fresh `"a"` = Add at the end

Professional Programming Rule

Always check `fp != NULL` before calling `fprintf()` or `fclose()` on the stream.

Reading One Character at a Time

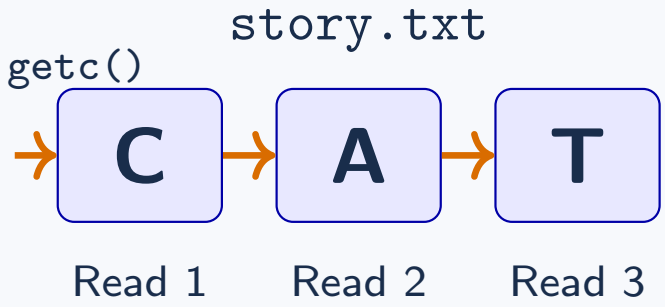


We know how to open a file. Now let's read its contents **one character at a time!**

Meet `getc()`

```
int getc(FILE *fp);
```

- `getc()` reads one character from a file stream.
- It returns the character that was read.
- It returns `EOF` when the end of input is reached or a read error occurs.



`getc()` picks up one character, then moves forward to the next character on the next call.

Let's Read story.txt!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Complete C Program

```
#include <stdio.h>
int main(void)
{
    FILE *fp = fopen("story.txt", "r");
    if (fp == NULL)
    {
        printf("Cannot open file.\n");
        return 1;
    }
    int ch;
    while ((ch = getc(fp)) != EOF)
    {
        putchar(ch);
    }
    fclose(fp);
    return 0;
}
```

Let's Read story.txt!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Complete C Program

```
#include <stdio.h>
int main(void)
{
    FILE *fp = fopen("story.txt", "r");
    if (fp == NULL)
    {
        printf("Cannot open file.\n");
        return 1;
    }
    int ch;
    while ((ch = getc(fp)) != EOF)
    {
        putchar(ch);
    }
    fclose(fp);
    return 0;
}
```

Input File

story.txt

CAT

Program Output

CAT

putc(ch) prints the character.

Why a Loop?

One call reads one character.
The loop repeats until EOF.

How Does the Loop Stop?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Consider the loop:

```
int ch;
while ((ch = getc(fp)) != EOF)
{
    putchar(ch);
}
```

Call	Value read	Action
1	'C'	Print C
2	'A'	Print A
3	'T'	Print T
4	EOF	Stop the loop

What Happens in the Condition?

1. `getc(fp)` reads one character.
2. The character is stored in `ch`.
3. If `ch != EOF`, the loop body runs.
4. If the result is `EOF`, the loop ends.

How Does the Loop Stop?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Consider the loop:

```
int ch;
while ((ch = getc(fp)) != EOF)
{
    putchar(ch);
}
```

Why Is the Return Type `int`?

How Does the Loop Stop?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Consider the loop:

```
int ch;
while ((ch = getc(fp)) != EOF)
{
    putchar(ch);
}
```

Why Is the Return Type `int`?

`getc()` must represent every possible `unsigned char` value **and** the special value `EOF`.

```
int ch;
```

Do not store the result in a `char` if you need to distinguish `EOF` reliably.

Why Use `int ch` Instead of `char ch`?



What must `getc()` return?

It must represent two kinds of values:

- **Real characters** read from the file.
- `EOF` — a special signal indicating that no more input is available or a read error occurred.

Real character values
`0, 1, 2, ..., 255`

Special value: `EOF`
Commonly `-1`

`int` can represent both!

Why not simply use `char ch`?

On a typical system with 8-bit bytes, an `unsigned char` can represent values from 0 to 255. But it cannot represent a negative `EOF` value. Even with a signed `char`, converting the result to `char` can make a real character indistinguishable from `EOF` on some implementations.

Trace the File Character by Character

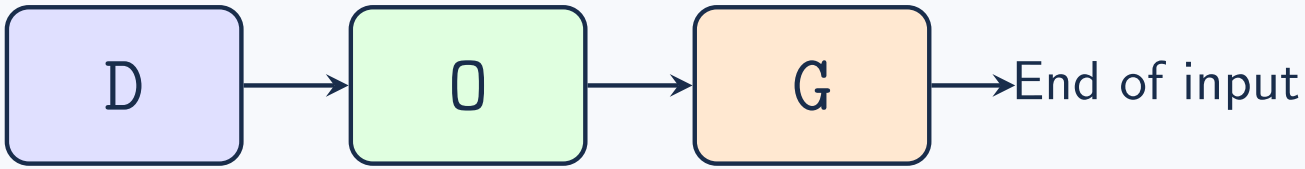
॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Suppose `story.txt` contains

DOG

Our program reads one character at a time:

```
int ch;
while ((ch = getc(fp)) != EOF)
{
    putchar(ch);
}
```



Remember

`getc(fp)` reads just one character per call. The loop continues until the returned value is `EOF`.

Follow Every Iteration

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Iteration	Character Read	Printed	Next Step
1	'D'	D	Continue
2	'O'	O	Continue
3	'G'	G	Continue
4	EOF	Nothing	Stop

What appears on the screen?

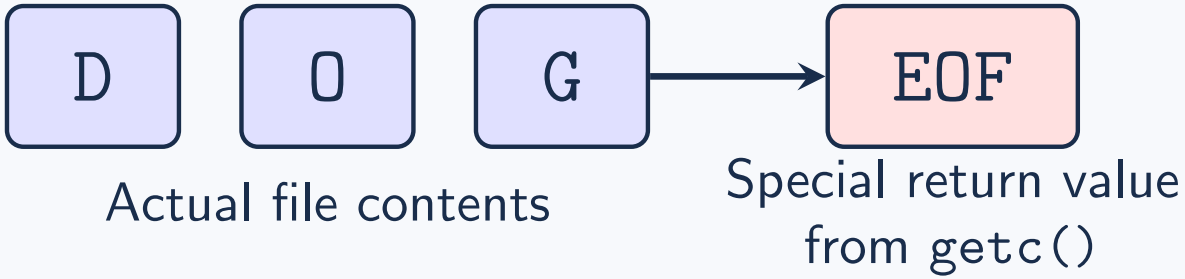
DOG

Why does the loop stop at iteration 4?

The fourth call to `getc(fp)` finds no more input and returns `EOF`. Therefore, `ch != EOF` becomes false.

Is EOF a Character in the File?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad



Important distinction

`EOF` is **not an ordinary character stored at the end of every file**. It is a special value returned by an input function when no more input is available or when a read error occurs.

Why is this useful?

The same loop can read files of different lengths without knowing in advance how many characters they contain.

End of File or Read Error?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Sometimes, we want to know **why a read operation failed**.

`feof(fp)`

Checks whether the stream's **end-of-file indicator** has been set.

Returns nonzero if set; otherwise returns zero.

`ferror(fp)`

Checks whether the stream's **error indicator** has been set.

Returns nonzero if set; otherwise returns zero.

When should we check them?

Check them after a read function indicates that reading has failed, for example, when `getc(fp)` returns `EOF`.

Key idea

`EOF` tells the loop to stop. Afterward, `ferror()` and `feof()` help identify the reason reading stopped.

Check EOF and Read Errors

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

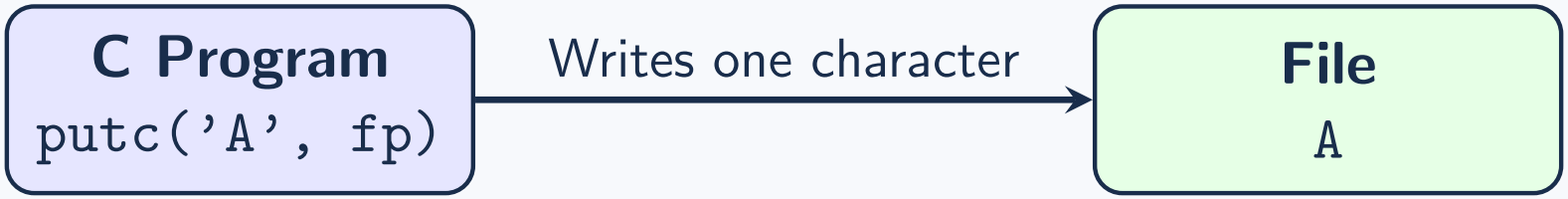
```
#include <stdio.h>
int main(void)
{
    FILE *fp = fopen("story.txt", "r");
    if (fp == NULL)
    {
        printf("Cannot open file.\n");
        return 1;
    }
    int ch;
    while ((ch = getc(fp)) != EOF)
    {       putchar(ch);    }
    if (ferror(fp))
        printf("\nA read error occurred.\n");
    else if (feof(fp))
        printf("\nEnd of file reached.\n");
    fclose(fp);
    return 0;}
```

putc() — Write One Character

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

What does putc() do?

putc() writes **one character** to an output stream, such as a file.



Syntax

```
putc(character, file_pointer);
```

- `character`: the character to write.
- `file_pointer`: the output stream, such as `fp`.

Write Characters into a File

Suppose we want to create a file named `message.txt` and write the word `CAT` into it.

Write Characters into a File

Suppose we want to create a file named `message.txt` and write the word `CAT` into it.

```
#include <stdio.h>
int main(void)
{
    FILE *fp = fopen("message.txt", "w");
    if (fp == NULL)
    {
        printf("Cannot open file.\n");
        return 1;
    }
    putc('C', fp);
    putc('A', fp);
    putc('T', fp);
    fclose(fp);
    return 0;
}
```

Write Characters into a File

Contents of `message.txt`

```
CAT
```

Notice

Each call to `putc()` writes one character. The mode `"w"` creates the file if needed, but erases its previous contents when opening an existing file succeeds.

What Does \n Do?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Consider these statements:

```
putc('C', fp);
putc('A', fp);
putc('T', fp);
putc('\n', fp);
putc('D', fp);
putc('O', fp);
putc('G', fp);
```

What Does \n Do?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Consider these statements:

```
putc('C', fp);
putc('A', fp);
putc('T', fp);
putc('\n', fp);
putc('D', fp);
putc('O', fp);
putc('G', fp);
```

Without the newline

```
CATDOG
```

With the newline

```
CAT
DOG
```

Remember

\n is the newline character. It moves the next output position to a new line; it is not the same as a space.

Did `putc()` Write Successfully?



Return value

- On success: returns the character written, as an `unsigned char` value converted to `int`.
- On a write error: returns `EOF`.

```
if (putc('A', fp) == EOF)
{
    printf("Write error!\n");
}
else
{
    printf("Character written.\n");
}
```

Why check the return value?

A write may fail, for example, because of a storage or device error. Checking the return value helps detect failure.
Also check the return value of `fclose(fp)` when you need to detect errors that occur while buffered output is being flushed.

Challenge — Copy One File into Another

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड

Your Challenge

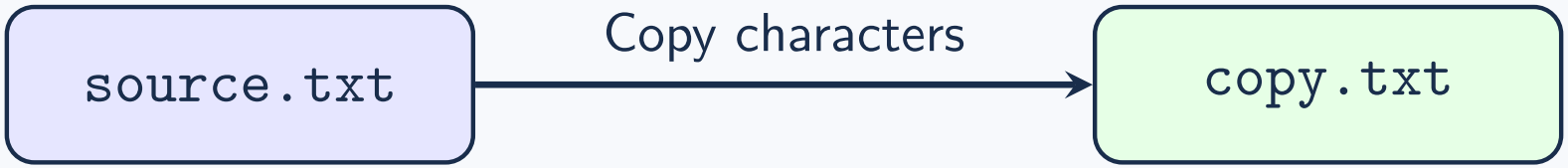
Write a C program to copy **all characters** from `source.txt` into `copy.txt`.

Input: `source.txt`

```
Hello
Welcome to C!
```

Output: `copy.txt`

```
Hello
Welcome to C!
```



Think first!

How can we copy a file without knowing how many characters it contains?

The File-Copying Algorithm

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

1. Open `source.txt` in read mode (`"r"`)
2. Open `copy.txt` in write mode (`"w"`)
3. Read one character using `getc()`
4. If not `EOF`, write it using `putc()`
5. Repeat steps 3–4 until `EOF`
6. Close both files

Key idea

Read one character, write that character, and repeat. Stop when the read operation returns `EOF`.

C Program to Copy

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

```
#include <stdio.h>
int main(void)
{
    FILE *src = fopen("source.txt", "r");
    if (src == NULL)
    {   printf("Cannot open source file.\n");
        return 1;
    }
    FILE *dst = fopen("copy.txt", "w");
    if (dst == NULL)
    {
        printf("Cannot open destination file.\n");
        fclose(src);
        return 1;
    }
```

C Program to Copy

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

```
    int ch;
    while ((ch = getc(src)) != EOF)
    {
        if (putc(ch, dst) == EOF)
        {
            printf("Write error!\n");
            fclose(src);
            fclose(dst);
            return 1;
        }
    }
    fclose(src);
    fclose(dst);
    printf("Copy operation finished.\n");
    return 0;
}
```

Trace the Copying Process

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Suppose `source.txt` contains `DOG`.

Read using `getc()`	**Write using** `putc()`	`copy.txt`
`'D'`	`'D'`	D
`'O'`	`'O'`	DO
`'G'`	`'G'`	DOG
`EOF`	Nothing	DOG

Final result

`source.txt` and `copy.txt` contain the same text.

What Could Go Wrong?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Challenge 1: What if `source.txt` does not exist?

`fopen("source.txt", "r")` returns `NULL`. The program should report the error and stop.

Challenge 2: What if writing fails?

`putc()` returns `EOF` on a write error. The program should report failure rather than claim success.

What Could Go Wrong?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Challenge 1: What if `source.txt` does not exist?

`fopen("source.txt", "r")` returns `NULL`. The program should report the error and stop.

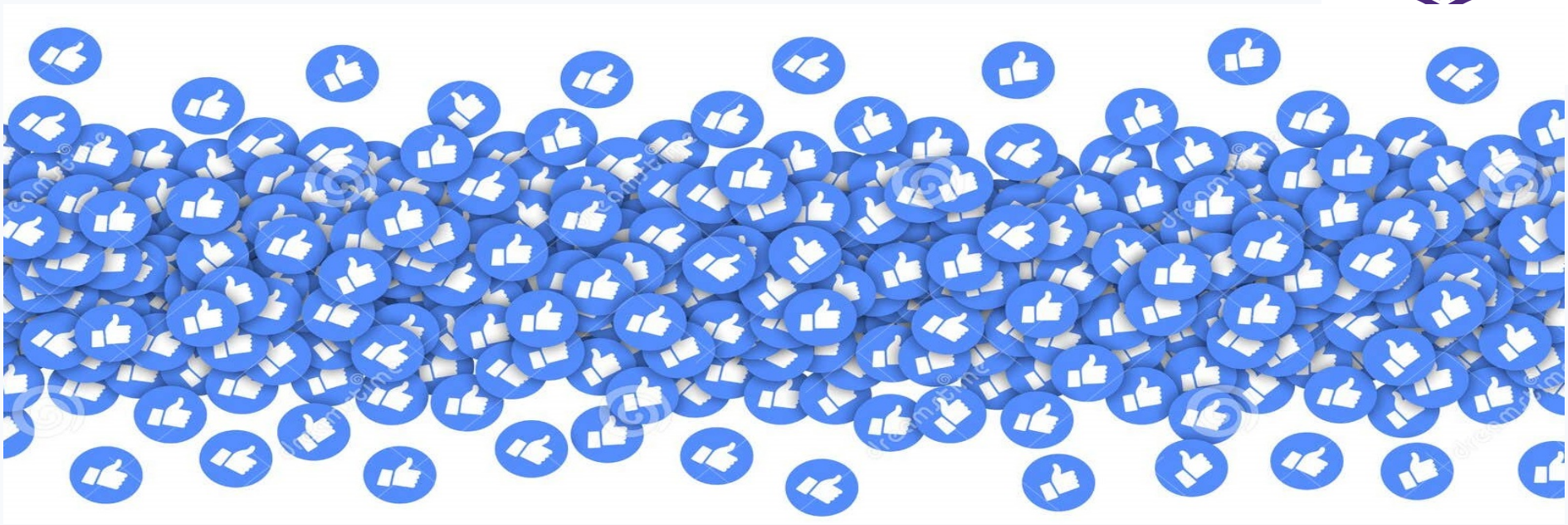
Challenge 2: What if writing fails?

`putc()` returns `EOF` on a write error. The program should report failure rather than claim success.

Challenge 3: What if both names refer to the same file?

Suppose `source.txt` and `copy.txt` refer to the **same file**.
Opening the destination using mode `"w"` can erase the file's existing contents before copying begins!

Correct code must also be safe code.



Thank you for your kind attention!!